Pertemuan 10
Klasifikasi dengan K-Nearest Neighbors

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Triyana Muliawati, S.Si., M.Si.
Rifky Fauzi

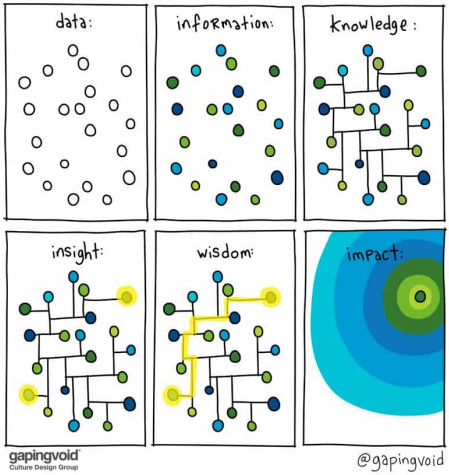
Program Studi Matematika
Institut Teknologi Sumatera

Semester Ganjil 2026/2027

Pokok bahasan

- Masalah klasifikasi dan representasi ruang fitur.
- KNN sebagai metode berbasis jarak dan tetangga lokal.
- Beberapa fungsi jarak dan pemilihan kelas dengan voting.
- Implementasi algoritma menggunakan NumPy.
- Normalisasi fitur dan evaluasi klasifikasi.
- Posisi KNN di antara model klasifikasi lain.

Data, Informasi, dan Keputusan



Ilustrasi: Gapingvoid Culture Design Group (gambar terlampir).

Data dapat dipandang sebagai kumpulan observasi. Setelah setiap observasi memiliki atribut dan label, kita dapat mengenali pola yang lebih terstruktur.

Masalah komputasi

Diberikan data berlabel dan sebuah objek baru, tentukan **kelas** objek baru tersebut berdasarkan informasi pada data yang tersedia.

Contoh: berdasarkan beberapa ukuran numerik, apakah suatu objek termasuk kelas A atau kelas B?

Pendekatan ini mengikuti pola pembelajaran komputasi yang konkret lebih dulu, lalu digeneralisasi ke bentuk matematis dan algoritmik.

Masalah Klasifikasi dan Variabel Masukan

Satu objek sebagai vektor

Jika satu objek mempunyai d atribut numerik, kita tulis

$$x = (x_1, x_2, \ldots, x_d) \in \mathbb{R}^d.$$

Setiap x_j disebut **fitur**. Untuk dua fitur,

$$x = (x_1, x_2)$$

dapat digambar sebagai satu titik pada bidang Kartesius.

Keluaran klasifikasi

Label kelas bersifat diskret, misalnya

$$y \in \{A, B\}$$

atau, secara umum,

$$y \in \{C_1, C_2, \ldots, C_m\}.$$

Untuk objek baru x^*, kita ingin menentukan prediksi kelas $\hat{y}(x^*)$.

Hubungan dengan matematika dasar

Geometri analitik memberi cara memandang data sebagai titik. Aljabar linear memberi cara menulis satu observasi sebagai vektor dan seluruh data sebagai matriks.

Klasifikasi dan Regresi

	Regresi	Klasifikasi
Masukan	$x \in \mathbb{R}^d$	$x \in \mathbb{R}^d$
Keluaran	nilai numerik $\hat{y} \in \mathbb{R}$	kelas $\hat{y} \in \{C_1, \ldots, C_m\}$
Contoh	harga, suhu, nilai ujian	lulus/tidak, spam/bukan spam
Pertanyaan	"berapa nilainya?"	"termasuk kelas mana?"

Catatan istilah

Nama suatu metode tidak selalu menunjukkan jenis keluarannya. **Logistic regression**, misalnya, justru lazim digunakan untuk klasifikasi biner karena keluarannya dapat diubah menjadi probabilitas kelas.

Representasi Data dalam Ruang Fitur

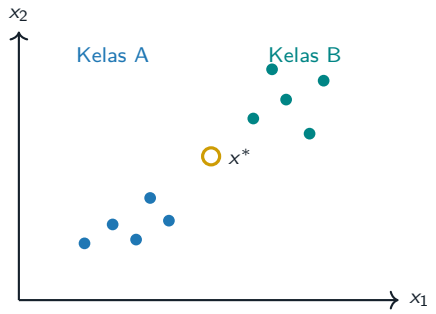
Data berlabel

Misalkan setiap mahasiswa direpresentasikan oleh

$$x = (x_1, x_2),$$

dengan x_1 = kehadiran dan x_2 = nilai tugas.
Label y menunjukkan dua kelas.
Untuk n observasi dan d fitur,

$$X = \begin{pmatrix} - - - x_1^T - - - \\ - - - x_2^T - - - \\ \vdots \\ - - - x_n^T - - - \end{pmatrix} \in \mathbb{R}^{n \times d}.$$



Masalah komputasi

Dari posisi x^* terhadap titik-titik berlabel, bagaimana aturan yang eksplisit untuk menentukan kelasnya?

Landasan Komputasi dari Dua Buku Rujukan

David J. Pine

Pada contoh jarak dua koordinat Kartesius, Pine menuliskan jarak dengan Teorema Pythagoras dan menghitungnya langsung menggunakan NumPy:

$$\Delta r = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}.$$

Ini menjadi landasan komputasi jarak pada KNN.

Wes McKinney

McKinney menempatkan *nearest neighbors*, logistic regression, SVM, dan random forest sebagai metode klasifikasi; ia juga menempatkan normalisasi sebagai bagian preprocessing. Pada contoh pemodelan, fitur disusun menjadi X_{train} dan label menjadi y_{train}.

Rujukan: Pine, *Introduction to Python for Science and Engineering*, §2.8.3; McKinney, *Python for Data Analysis*, 3rd ed., §1.3 dan §12.4.

Prinsip K-Nearest Neighbors

Diberikan data latih

$$D = \{(x_i, y_i)\}_{i=1}^{n}, \qquad x_i \in \mathbb{R}^d.$$

Untuk objek baru x^*, KNN menggunakan informasi kelas dari k observasi yang **paling dekat** dengan x^*.

Tiga komponen utama

1. fungsi jarak $d(x_i, x^*)$;
2. jumlah tetangga k;
3. aturan voting kelas.

Interpretasi geometri

Di sekitar x^* dibentuk lingkungan lokal. Kelas diputuskan dari komposisi label di lingkungan tersebut, bukan dari satu persamaan garis global.

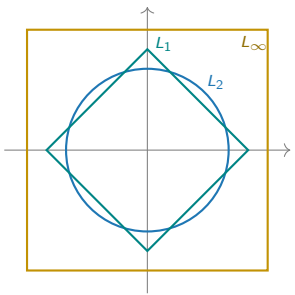
KNN adalah metode lokal

Dua objek yang berdekatan di ruang fitur dianggap lebih serupa daripada dua objek yang berjauhan, sejauh definisi jarak yang dipakai memang sesuai dengan masalah.

Beberapa Fungsi Jarak pada Ruang Fitur

Jarak	Rumus
Euclidean (L_2)	$d_2(x,z) = \sqrt{\sum_j (x_j - z_j)^2}$
Manhattan (L_1)	$d_1(x,z) = \sum_j \lvert x_j - z_j \rvert$
Chebyshev (L_∞)	$d_\infty(x,z) = \max_j \lvert x_j - z_j \rvert$
Minkowski (L_p)	$d_p(x,z) = \left(\sum_j \lvert x_j - z_j \rvert^p \right)^{1/p}$

Bentuk titik berjarak 1 dari pusat



Pemilihan jarak adalah bagian dari model

Jarak Euclidean cocok untuk geometri Kartesius biasa.
Masalah lain dapat membutuhkan ukuran jarak yang berbeda.

Makna geometri

Fungsi jarak yang berbeda menghasilkan bentuk "lingkungan terdekat" yang berbeda.

Jarak pada Koordinat Geografis

Lintang dan bujur bukan bidang Kartesius biasa

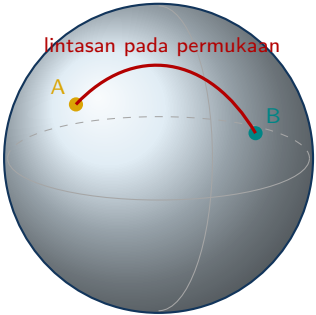
Untuk dua lokasi di permukaan bumi, selisih lintang-bujur tidak sebaiknya langsung diperlakukan sebagai jarak Euclidean datar, terutama untuk wilayah luas.

Jarak lintasan terpendek di permukaan bola

Jika R adalah jari-jari bumi dan $\Delta\sigma$ sudut pusat, maka

$$d = R\,\Delta\sigma.$$

Salah satu cara menghitung $\Delta\sigma$ adalah rumus Haversine.



Contoh aplikasi

Jika fitur KNN adalah posisi geografis, pilihan fungsi jarak harus mengikuti geometri permukaan bumi.

Aturan Keputusan Berdasarkan Voting

Setelah k tetangga terdekat ditemukan, kelas ditentukan dari jumlah suara.

Contoh tiga kelas

Misalkan $k = 7$ dan label tetangga adalah

$$A,\ B,\ A,\ C,\ A,\ B,\ A.$$

Maka

$$v_A = 4, \qquad v_B = 2, \qquad v_C = 1.$$

Karena v_A paling besar, prediksi kelas adalah A.

Secara umum

Untuk setiap kelas C_j, hitung

$$v_j = \text{jumlah tetangga yang berlabel } C_j.$$

Kemudian pilih kelas dengan nilai v_j terbesar.

Tidak perlu fungsi indikator dulu

Pada tahap ini cukup pahami operasi elementer: menghitung frekuensi setiap kelas, lalu mencari nilai maksimum.

Fungsi `max` dan Notasi argmax

Perbedaan sederhana

Misalkan suara tiga kelas adalah

$$(v_A, v_B, v_C) = (2, 5, 3).$$

- $\max(2, 5, 3) = 5$ memberi **nilai terbesar**.
- $\text{argmax}(2, 5, 3) = B$ memberi **kelas tempat nilai terbesar berada**.

Notasi KNN

Jika v_j adalah jumlah suara untuk kelas C_j, tulis

$$r = \underset{j=1,\ldots,m}{\arg\max}\, v_j, \qquad \hat{y} = C_r.$$

Artinya: cari indeks kelas dengan suara terbesar, kemudian ambil nama kelas tersebut.

Hubungan dengan Python

Jika `jumlah = [2, 5, 3]`, maka `np.argmax(jumlah)` menghasilkan indeks posisi nilai terbesar.

Algoritma KNN untuk Klasifikasi

Masukan

Data latih (X, y), objek baru x^*, fungsi jarak, dan jumlah tetangga k.

1. Hitung jarak $d_i = d(x_i, x^*)$ untuk setiap observasi $i = 1, \ldots, n$.
2. Urutkan jarak dari yang terkecil ke terbesar.
3. Ambil k indeks dengan jarak terkecil.
4. Ambil label dari k observasi tersebut.
5. Hitung jumlah suara setiap kelas.
6. Pilih kelas dengan suara terbanyak.

Jejak algoritma yang harus dapat dijelaskan

Nilai jarak $\rightarrow$ urutan indeks $\rightarrow$ k tetangga terpilih $\rightarrow$ jumlah suara $\rightarrow$ kelas akhir.

Contoh Perhitungan KNN

Gunakan objek baru $x^* = (3.1, 2.7)$, jarak Euclidean, dan $k = 3$.

Objek	Titik	Kelas	$d(x_i, x^*)$
1	$(1,1)$	A	2.70
2	$(2,1)$	A	2.03
3	$(2,2)$	A	1.30
4	$(4,4)$	B	1.58
5	$(5,4)$	B	2.30
6	$(4,5)$	B	2.47

Tiga jarak terkecil:

objek 3 (A), 4 (B), dan 2 (A).

Voting

$$v_A = 2, \qquad v_B = 1.$$

Karena $v_A > v_B$,

$$\boxed{\hat{y}(x^*) = A}.$$

Yang berubah jika k berubah

Jarak seluruh data tetap sama. Yang berubah adalah banyak tetangga yang ikut memberikan suara.

Pemilihan Nilai k

$k = 1$

Keputusan sangat lokal. Satu observasi yang salah label atau pencilan dapat langsung mengubah prediksi.

k sedang

Beberapa tetangga ikut memberikan suara sehingga keputusan biasanya lebih stabil.

k terlalu besar

Observasi yang jauh ikut memengaruhi keputusan dan pola lokal dapat hilang.

Istilah penting

k bukan koefisien yang diestimasi dari persamaan model. k adalah **hyperparameter**: nilai yang kita pilih dan kemudian evaluasi menggunakan data validasi atau prosedur cross-validation.

KNN sebagai Metode Nonparametrik

Regresi linier

Kita mengestimasi koefisien global

$$\beta_0, \beta_1, \ldots, \beta_d$$

agar satu persamaan berlaku untuk seluruh ruang fitur.

Logistic regression

Kita juga mengestimasi koefisien β, lalu membentuk probabilitas kelas melalui fungsi sigmoid.

KNN

Tidak ada koefisien global seperti $\beta_0, \beta_1, \ldots$ yang harus dicari terlebih dahulu. Data latih disimpan, kemudian dipakai langsung saat objek baru datang.

Bahasa sederhana

KNN tidak "meringkas" seluruh data menjadi satu persamaan dengan beberapa koefisien. Karena itu KNN disebut **nonparametrik**. Namun k, fungsi jarak, dan cara penskalaan fitur tetap harus dipilih.

Implementasi Algoritma KNN dengan NumPy

```python
import numpy as np

def knn_predict(X, y, x_new, k=3):
    # 1. hitung jarak Euclidean
    diff = X - x_new
    jarak = np.sqrt(np.sum(diff**2, axis=1))

    # 2. ambil k jarak terkecil
    idx = np.argsort(jarak)[:k]
    tetangga = y[idx]

    # 3. hitung suara setiap kelas
    kelas = np.unique(tetangga)
    suara = []
    for c in kelas:
        suara.append(np.sum(tetangga == c))

    # 4. pilih kelas dengan suara maksimum
    prediksi = kelas[np.argmax(suara)]
    return prediksi, jarak, idx, tetangga
```

Operasi yang digunakan

- pengurangan vektor;
- kuadrat, jumlah, akar;
- sorting indeks;
- perulangan dan counting;
- indeks maksimum.

Fokus perkuliahan

NumPy membantu menjalankan operasi array; algoritma klasifikasinya tetap terlihat dan dapat ditelusuri.

Kompleksitas Algoritma KNN

Misalkan terdapat n observasi latih dan d fitur. Untuk implementasi yang memakai `argsort`:

Langkah	Operasi	Big-O
Hitung jarak	$n \times d$ komponen	$O(nd)$
Urutkan jarak	sorting n nilai	$O(n \log n)$
Ambil tetangga	k indeks	$O(k)$
Voting	k label	$O(k)$

Total satu prediksi

$$T(n, d, k) = O(nd + n \log n + k).$$

Jika d dan k dianggap jauh lebih kecil daripada n,

$$T(n) = O(n \log n).$$

Konsekuensi

Tahap "belajar" KNN ringan, tetapi prediksi membutuhkan akses ke data latih dan perhitungan jarak.

Normalisasi Min-Max pada Fitur

Masalah skala

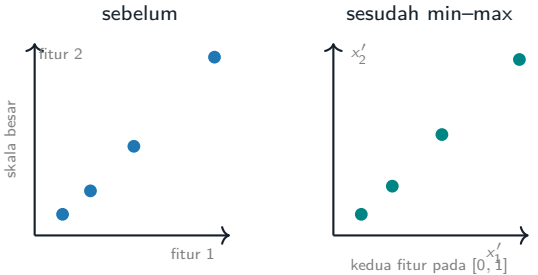
Misalkan fitur pertama berada pada rentang 0–10, sedangkan fitur kedua berada pada rentang 0–1000. Pada jarak Euclidean, perubahan pada fitur kedua dapat mendominasi hanya karena satuannya lebih besar.

Min-max normalization

Untuk fitur ke-j:

$$x'_{ij} = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)}.$$

Hasilnya berada pada rentang $[0, 1]$.



Tujuan

Bukan membuat data “lebih bagus”, tetapi membuat kontribusi jarak antarfitur menjadi lebih sebanding.

Standardisasi sebagai Alternatif Penskalaan

Z-score standardization

Alternatif lain adalah

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j},$$

dengan μ_j rata-rata dan s_j simpangan baku fitur ke-j.

Perbedaan intuitif

- Min-max: memetakan rentang menjadi $[0, 1]$.
- Z-score: memusatkan data di sekitar 0 dan mengukur posisi dalam satuan simpangan baku.

Aturan penting pada train/test

Nilai minimum, maksimum, rata-rata, atau simpangan baku harus dihitung dari **data latih**. Transformasi yang sama kemudian diterapkan pada data uji dan objek baru.

Mengapa?

Agar informasi dari data uji tidak "bocor" ke tahap pembentukan model atau preprocessing.

Data Latih dan Data Uji

Data latih

Dipakai sebagai observasi berlabel yang menjadi referensi tetangga KNN.

Data uji

Dipakai untuk menilai apakah aturan klasifikasi bekerja pada data yang tidak digunakan sebagai referensi saat prediksi dibentuk.

Urutan yang aman

1. pisahkan train dan test;
2. hitung parameter penskalaan dari train;
3. transformasikan train dan test;
4. lakukan prediksi pada test;
5. bandingkan prediksi dengan label sebenarnya.

McKinney mencontohkan pola umum: model dibentuk pada training data dan dievaluasi pada out-of-sample testing data.

Confusion Matrix untuk Klasifikasi Biner

Untuk satu kelas yang ditetapkan sebagai **positif**, hasil prediksi dapat diringkas sebagai berikut.

	Prediksi positif	Prediksi negatif
Aktual positif	TP (true positive)	FN (false negative)
Aktual negatif	FP (false positive)	TN (true negative)

Kesalahan tipe pertama dalam tabel

FP: model menyatakan positif padahal aktual negatif.

Kesalahan tipe kedua dalam tabel

FN: model menyatakan negatif padahal aktual positif.

Mengapa tidak cukup akurasi?

Dua model dengan akurasi sama dapat mempunyai komposisi FP dan FN yang sangat berbeda.

Ukuran Kinerja Klasifikasi

Dengan notasi pada confusion matrix:

Accuracy dan Precision

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Specificity

$$\text{Specificity} = \frac{TN}{TN + FP}.$$

F1-score

$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Pemilihan ukuran bergantung pada masalah

Jika false negative sangat berbahaya, recall menjadi penting. Jika false positive mahal, precision perlu diperhatikan.

Beberapa Model untuk Klasifikasi

Metode	Gagasan utama	Apa yang dipelajari dari data?
KNN	tetangga lokal menurut jarak	tidak mengestimasi koefisien global; menyimpan data latih
Logistic regression	skor linier diubah menjadi probabilitas	koefisien β pada fungsi linier
Decision tree	aturan pemisahan bertahap	titik/aturan pemisahan fitur
Random forest	banyak decision tree	kumpulan pohon dari sampel/fitur berbeda
SVM	batas kelas dengan margin	batas keputusan yang memaksimalkan margin
Naive Bayes	probabilitas bersyarat	parameter distribusi/probabilitas tiap kelas

Contoh logistic regression untuk dua kelas

Bentuk skor liniernya $s(x) = \beta_0 + \beta^T x$. Probabilitas kelas 1 dapat ditulis

$$p(y = 1 \mid x) = \frac{1}{1 + e^{-s(x)}}.$$

Koefisien β diestimasi dari data; ini berbeda dari KNN yang memakai tetangga lokal saat prediksi.

Latihan: KNN dari Geometri hingga Evaluasi

Gunakan data berikut.

No	x_1	x_2	Kelas
1	1	1	A
2	2	1	A
3	2	2	A
4	3	2	A
5	3	3	A
6	5	4	B
7	5	5	B
8	6	4	B
9	6	5	B
10	7	5	B

Tugas

1. Untuk $x^* = (4.2, 3.2)$, hitung jarak Euclidean ke seluruh data.
2. Tentukan prediksi untuk $k = 3$ dan $k = 5$.
3. Implementasikan fungsi KNN menggunakan NumPy tanpa pustaka pemodelan.
4. Tampilkan indeks tetangga, label, dan jumlah suara setiap kelas.
5. Tuliskan $T(n, d, k)$ dan Big-O implementasi.
6. Buat satu contoh fitur dengan skala berbeda, lalu lakukan min-max normalization sebelum KNN.

Ringkasan

Konsep matematis

- observasi sebagai vektor $x \in \mathbb{R}^d$;
- fungsi jarak menentukan arti “dekat”;
- voting menentukan kelas dari tetangga lokal;
- penskalaan fitur memengaruhi hasil jarak.

Konsep komputasi

- operasi array dan jarak;
- sorting indeks dan counting label;
- kompleksitas satu prediksi;
- train/test, confusion matrix, dan metrik evaluasi.

Posisi KNN

KNN adalah metode klasifikasi nonparametrik yang sederhana secara konsep, tetapi tetap memerlukan keputusan matematis penting: representasi fitur, fungsi jarak, nilai k, dan penskalaan.

Rujukan

David J. Pine. *Introduction to Python for Science and Engineering*. CRC Press, 2019. Lihat terutama §2.8.3 untuk jarak Kartesius dengan NumPy dan bab tentang array/linear algebra.

Wes McKinney. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*, 3rd ed. O'Reilly, 2022. Lihat §1.3 dan §12.4 untuk ekosistem model klasifikasi, preprocessing, serta train/test.

Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani. *An Introduction to Statistical Learning*, 2nd ed. Springer, 2021.

Trevor Hastie, Robert Tibshirani, Jerome Friedman. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.